



INSTITUTO POLITÉCNICO NACIONAL

Escuela Superior de Cómputo



Carrera:

Ingeniería en Sistemas Computacionales

Unidad de Aprendizaje:

Análisis y Diseño de Algoritmos

Actividad:

Práctica 1.1 Medición del tiempo en C

Nombre de los Alumnos:

Álvarez Tahuilán Luis Gustavo

Noyola Gómez Emilio Damian

Nombre del Maestro:

Ricardo Felipe Díaz Santiago

Turno: Matutino

Grupo: 3CM1

Fecha de Entrega: 09 de septiembre de 2026

Práctica 1.1 Medición del tiempo en C

Introducción

El análisis de algoritmos permite estudiar el comportamiento de un programa en función del tamaño de los datos de entrada y determinar qué tan eficiente resulta conforme dicho tamaño aumenta. En esta práctica se analiza el algoritmo de Ordenamiento por Selección **Selection Sort**, un método que recorre repetidamente un arreglo para localizar el elemento menor de la parte no ordenada y colocarlo en la posición correspondiente.

Para evaluar su comportamiento, el algoritmo se implementó en lenguaje C y se probó con arreglos de diferentes tamaños bajo cuatro condiciones iniciales: datos aleatorios, orden inverso, ordenados y casi ordenados. En cada caso se midió el tiempo de ejecución con el propósito de observar experimentalmente cómo aumenta el costo temporal al incrementar la cantidad de elementos.

Además del análisis experimental, se realiza un análisis analítico mediante el conteo de instrucciones y pasos del algoritmo, con el fin de obtener su función de costo, determinar su orden de ejecución en el mejor y peor caso y comparar los resultados teóricos con los obtenidos durante las pruebas. De esta forma, se busca comprobar el comportamiento cuadrático característico del algoritmo de Ordenamiento por Selección.

El objetivo final es relacionar el análisis teórico del algoritmo con su comportamiento experimental y verificar si los tiempos obtenidos corresponden con el orden de crecimiento esperado.

Desarrollo

I. Explicación del funcionamiento

```
25 void selectionSort(int arreglo[], int n)
26 {
27     for (int i = 0; i < n - 1; i++)
28     {
29         int posicionMenor = i;
30
31         for (int j = i + 1; j < n; j++)
32         {
33             if (arreglo[j] < arreglo[posicionMenor])
34             {
35                 posicionMenor = j;
36             }
37         }
38
39         if (posicionMenor != i)
40         {
41             int auxiliar = arreglo[i];
42             arreglo[i] = arreglo[posicionMenor];
43             arreglo[posicionMenor] = auxiliar;
44         }
45     }
46 }
```

Figura 1. Esta parte contiene el algoritmo principal de **Selection Sort**; recorre el arreglo, busca el elemento

más pequeño dentro de la parte que todavía no está ordenada y lo intercambia con la posición actual. Este proceso se repite hasta que todos los elementos quedan acomodados de menor a mayor. Además, es la sección principal que se utiliza para medir el tiempo de ejecución y analizar la *complejidad* del algoritmo.

```
49 void generarAleatorio(int arreglo[], int n)
50 {
51     for (int i = 0; i < n; i++)
52     {
53         arreglo[i] = rand();
54     }
55 }
56
57
58 void invertirArreglo(int arreglo[], int n)
59 {
60     for (int i = 0; i < n / 2; i++)
61     {
62         int auxiliar = arreglo[i];
63
64         arreglo[i] = arreglo[n - 1 - i];
65         arreglo[n - 1 - i] = auxiliar;
66     }
67 }
```

Figura 2. Esta parte prepara algunas de las condiciones de prueba del experimento: la función *generarAleatorio* llena el arreglo con valores generados mediante *rand()*, mientras que *invertirArreglo* cambia el orden de los elementos para obtener un arreglo en orden inverso. Estas funciones permiten crear distintas entradas sobre las que después se ejecuta *Selection Sort*, con el fin de comparar su comportamiento y tiempo de ejecución bajo diferentes condiciones.

```

70 void casiOrdenar(int arreglo[], int n)
71 {
72     int posiciones[ELEMENTOS_MOVIDOS];
73
74     for (int i = 0; i < ELEMENTOS_MOVIDOS; i++)
75     {
76         int posicion;
77         int repetida;
78
79         do
80         {
81             posicion = rand() % n;
82             repetida = 0;
83
84             for (int j = 0; j < i; j++)
85             {
86                 if (posiciones[j] == posicion)
87                 {
88                     repetida = 1;
89                     break;
90                 }
91             }
92         } while (repetida);
93
94         posiciones[i] = posicion;
95     }
96
97     int auxiliar = arreglo[posiciones[ELEMENTOS_MOVIDOS - 1]];
98
99     for (int i = ELEMENTOS_MOVIDOS - 1; i > 0; i--)
100     {
101         arreglo[posiciones[i]] = arreglo[posiciones[i - 1]];
102     }
103
104     arreglo[posiciones[0]] = auxiliar;
105 }
106

```

Figura 3. Esta función genera el caso de prueba casi ordenado. Parte de un arreglo que ya está **ordenado**, selecciona 10 posiciones diferentes de manera aleatoria y mueve los valores entre esas posiciones, evitando repetir índices. De esta forma se obtiene un arreglo que conserva casi todo su orden original, pero con algunos elementos fuera de lugar, permitiendo evaluar cómo se comporta **Selection Sort** cuando la entrada está muy cerca de estar completamente ordenada.

```

123 void ejecutarPrueba(int arreglo[], int n, const char *condicion)
124 {
125     printf("\n-----\n");
126     printf("Tamano: %d\n", n);
127     printf("Condicion: %s\n", condicion);
128
129     clock_t inicio = clock();
130
131     selectionSort(arreglo, n);
132
133     clock_t fin = clock();
134
135     double tiempo = (double)(fin - inicio) / CLOCKS_PER_SEC;
136
137     int correcto = estaOrdenado(arreglo, n);
138
139     printf("Tiempo: %.6f segundos\n", tiempo);
140     printf("Ordenamiento correcto: %s\n", correcto ? "SI" : "NO");
141 }

```

Figura 4. Esta función se encarga de ejecutar y medir cada prueba del algoritmo. Recibe el arreglo, su tamaño y la condición evaluada, registra el tiempo antes y después de llamar a **Selection Sort**, calcula cuánto tardó la

ejecución en segundos y verifica que el arreglo haya quedado correctamente ordenado. También muestra estos resultados en consola para poder comparar el comportamiento del algoritmo entre las distintas pruebas.

```
190 // Caso 1:
191 generarAleatorio(base, n);
192 memcpy(trabajo, base, (size_t)n * sizeof(int));
193 ejecutarPrueba(trabajo, n, "Aleatorio");
194
195 // Ordenar arreglo
196 qsort(base, n, sizeof(int), compararEnteros);
197
198
199 // Caso 2:
200 memcpy(trabajo, base, (size_t)n * sizeof(int));
201 invertirArreglo(trabajo, n);
202 ejecutarPrueba(trabajo, n, "Inverso");
203
204
205 // Caso 3:
206 memcpy(trabajo, base, (size_t)n * sizeof(int));
207 ejecutarPrueba(trabajo, n, "Ordenado");
208
209
210 // Caso 4:
211 memcpy(trabajo, base, (size_t)n * sizeof(int));
212 casiOrdenar(trabajo, n);
213 ejecutarPrueba(trabajo, n, "Casi ordenado");
```

Figura 5. Esta parte organiza la ejecución de los cuatro casos de prueba: primero genera un arreglo aleatorio, después prepara una versión en orden inverso, una ordenada y otra casi ordenada. En cada caso se copia el arreglo base para no modificar los datos originales y posteriormente se llama a `ejecutarPrueba`, que mide el tiempo de `Selection Sort` para cada condición.

II. Resultado de las Pruebas

```
=====
PRUEBAS PARA N = 100
=====

-----
Tamano: 100
Condicion: Aleatorio
Tiempo: 0.000000 segundos
Ordenamiento correcto: SI

-----
Tamano: 100
Condicion: Inverso
Tiempo: 0.000000 segundos
Ordenamiento correcto: SI

-----
Tamano: 100
Condicion: Ordenado
Tiempo: 0.000000 segundos
Ordenamiento correcto: SI

-----
Tamano: 100
Condicion: Casi ordenado
Tiempo: 0.000000 segundos
Ordenamiento correcto: SI
```

Figura 5. Pruebas con 100 elementos en las cuatro condiciones.

```
=====
PRUEBAS PARA N = 500
=====

-----
Tamano: 500
Condicion: Aleatorio
Tiempo: 0.000000 segundos
Ordenamiento correcto: SI

-----
Tamano: 500
Condicion: Inverso
Tiempo: 0.000000 segundos
Ordenamiento correcto: SI

-----
Tamano: 500
Condicion: Ordenado
Tiempo: 0.000000 segundos
Ordenamiento correcto: SI

-----
Tamano: 500
Condicion: Casi ordenado
Tiempo: 0.000000 segundos
Ordenamiento correcto: SI
```

Figura 6. Pruebas con 500 elementos en las cuatro condiciones.

```
=====
PRUEBAS PARA N = 1500
=====

-----
Tamano: 1500
Condicion: Aleatorio
Tiempo: 0.000000 segundos
Ordenamiento correcto: SI

-----
Tamano: 1500
Condicion: Inverso
Tiempo: 0.005000 segundos
Ordenamiento correcto: SI

-----
Tamano: 1500
Condicion: Ordenado
Tiempo: 0.002000 segundos
Ordenamiento correcto: SI

-----
Tamano: 1500
Condicion: Casi ordenado
Tiempo: 0.001000 segundos
Ordenamiento correcto: SI
```

Figura 7. Pruebas con 1500 elementos en las cuatro condiciones.

```

=====
PRUEBAS PARA N = 5000
=====

-----

Tamano: 5000
Condicion: Aleatorio
Tiempo: 0.013000 segundos
Ordenamiento correcto: SI

-----

Tamano: 5000
Condicion: Inverso
Tiempo: 0.020000 segundos
Ordenamiento correcto: SI

-----

Tamano: 5000
Condicion: Ordenado
Tiempo: 0.008000 segundos
Ordenamiento correcto: SI

-----

Tamano: 5000
Condicion: Casi ordenado
Tiempo: 0.018000 segundos
Ordenamiento correcto: SI

```

Figura 8. Pruebas con 5000 elementos en las cuatro condiciones.

```

=====
PRUEBAS PARA N = 10000
=====

-----

Tamano: 10000
Condicion: Aleatorio
Tiempo: 0.043000 segundos
Ordenamiento correcto: SI

-----

Tamano: 10000
Condicion: Inverso
Tiempo: 0.072000 segundos
Ordenamiento correcto: SI

-----

Tamano: 10000
Condicion: Ordenado
Tiempo: 0.037000 segundos
Ordenamiento correcto: SI

-----

Tamano: 10000
Condicion: Casi ordenado
Tiempo: 0.036000 segundos
Ordenamiento correcto: SI

```

Figura 11. Pruebas con 10000 elementos en las cuatro condiciones.

```

=====
PRUEBAS PARA N = 15000
=====

-----

Tamano: 15000
Condicion: Aleatorio
Tiempo: 0.080000 segundos
Ordenamiento correcto: SI

-----

Tamano: 15000
Condicion: Inverso
Tiempo: 0.112000 segundos
Ordenamiento correcto: SI

-----

Tamano: 15000
Condicion: Ordenado
Tiempo: 0.108000 segundos
Ordenamiento correcto: SI

-----

Tamano: 15000
Condicion: Casi ordenado
Tiempo: 0.079000 segundos
Ordenamiento correcto: SI

```

Figura 9. Pruebas con 15000 elementos en las cuatro condiciones.

```

=====
PRUEBAS PARA N = 50000
=====

-----

Tamano: 50000
Condicion: Aleatorio
Tiempo: 0.901000 segundos
Ordenamiento correcto: SI

-----

Tamano: 50000
Condicion: Inverso
Tiempo: 2.373000 segundos
Ordenamiento correcto: SI

-----

Tamano: 50000
Condicion: Ordenado
Tiempo: 0.847000 segundos
Ordenamiento correcto: SI

-----

Tamano: 50000
Condicion: Casi ordenado
Tiempo: 0.870000 segundos
Ordenamiento correcto: SI

```

Figura 10. Pruebas con 50000 elementos en las cuatro condiciones.

```

=====
PRUEBAS PARA N = 100000
=====

-----
Tamano: 100000
Condicion: Aleatorio
Tiempo: 4.105000 segundos
Ordenamiento correcto: SI

-----
Tamano: 100000
Condicion: Inverso
Tiempo: 19.974000 segundos
Ordenamiento correcto: SI

-----
Tamano: 100000
Condicion: Ordenado
Tiempo: 7.728000 segundos
Ordenamiento correcto: SI

-----
Tamano: 100000
Condicion: Casi ordenado
Tiempo: 7.553000 segundos
Ordenamiento correcto: SI

```

Figura 12. Pruebas con 100000 elementos en las cuatro condiciones.

```

=====
PRUEBAS PARA N = 250000
=====

-----
Tamano: 250000
Condicion: Aleatorio
Tiempo: 40.378000 segundos
Ordenamiento correcto: SI

-----
Tamano: 250000
Condicion: Inverso
Tiempo: 74.164000 segundos
Ordenamiento correcto: SI

-----
Tamano: 250000
Condicion: Ordenado
Tiempo: 48.118000 segundos
Ordenamiento correcto: SI

-----
Tamano: 250000
Condicion: Casi ordenado
Tiempo: 44.393000 segundos
Ordenamiento correcto: SI

```

Figura 14. Pruebas con 250000 elementos en las cuatro condiciones.

```

=====
PRUEBAS PARA N = 500000
=====

-----
Tamano: 500000
Condicion: Aleatorio
Tiempo: 196.133000 segundos
Ordenamiento correcto: SI

-----
Tamano: 500000
Condicion: Inverso
Tiempo: 170.418000 segundos
Ordenamiento correcto: SI

-----
Tamano: 500000
Condicion: Ordenado
Tiempo: 143.247000 segundos
Ordenamiento correcto: SI

-----
Tamano: 500000
Condicion: Casi ordenado
Tiempo: 111.933000 segundos
Ordenamiento correcto: SI

```

Figura 13. Pruebas con 500000 elementos en las cuatro condiciones.

```

=====
PRUEBAS PARA N = 1000000
=====

-----
Tamano: 1000000
Condicion: Aleatorio
Tiempo: 747.330000 segundos
Ordenamiento correcto: SI

-----
Tamano: 1000000
Condicion: Inverso
Tiempo: 905.565000 segundos
Ordenamiento correcto: SI

-----
Tamano: 1000000
Condicion: Ordenado
Tiempo: 791.400000 segundos
Ordenamiento correcto: SI

-----
Tamano: 1000000
Condicion: Casi ordenado
Tiempo: 647.758000 segundos
Ordenamiento correcto: SI

```

Figura 15. Pruebas con 1000000 elementos en las cuatro condiciones.

III. Gráficas de Pruebas

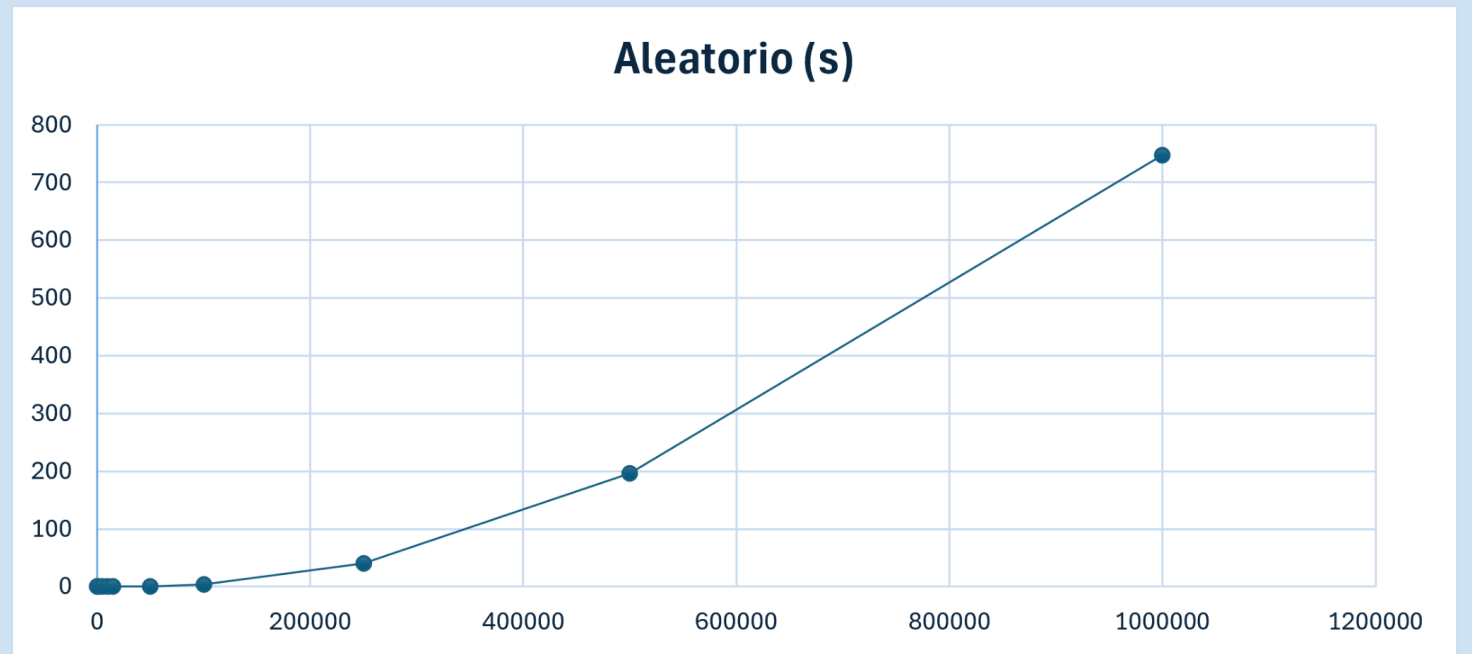


Figura 16. Esta gráfica muestra el tiempo de ejecución de Selection Sort con arreglos en orden aleatorio para distintos tamaños. Se observa que, conforme aumenta la cantidad de elementos, el tiempo crece cada vez más rápido, formando una curva característica de un comportamiento cuadrático.



Figura 17. Esta gráfica muestra el tiempo de ejecución de Selection Sort cuando el arreglo ya se encuentra

ordenado. Aunque los datos están en su mejor condición inicial, el tiempo sigue aumentando de forma considerable conforme crece el tamaño del arreglo, ya que el algoritmo continúa recorriendo la parte restante para buscar el elemento menor.

Inverso (s)

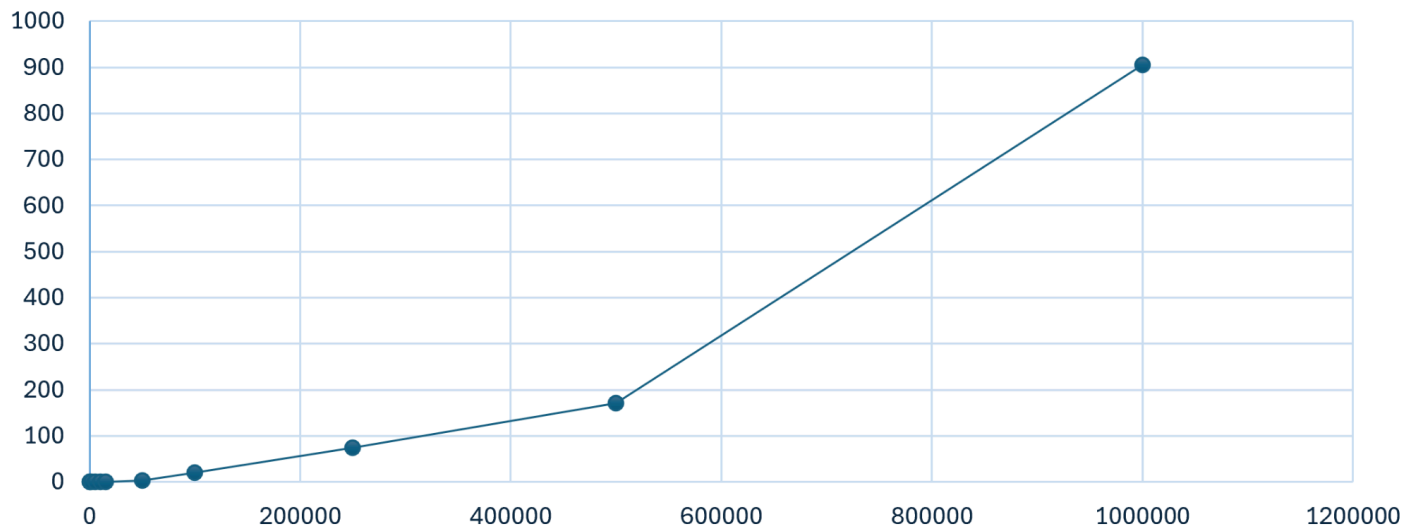


Figura 18. Esta gráfica muestra el tiempo de ejecución de Selection Sort cuando el arreglo se encuentra en orden inverso. Se observa que el tiempo aumenta de forma acelerada conforme crece el número de elementos, alcanzando los valores más altos entre las condiciones probadas. Este comportamiento es consistente con el análisis del algoritmo, ya que sigue realizando una cantidad de operaciones proporcional a n^2 .

Casi ordenado (s)

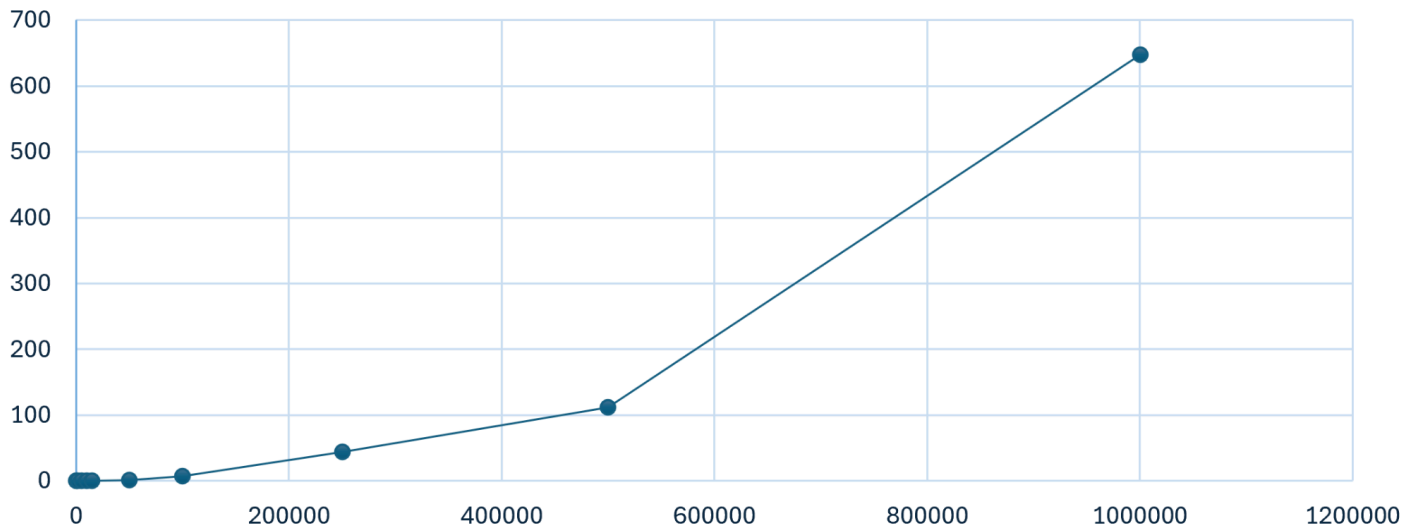


Figura 19. Esta gráfica muestra el tiempo de ejecución de Selection Sort con un arreglo casi ordenado, es decir, con pocos elementos fuera de su posición correcta. Aunque esta condición presenta tiempos menores que otros casos para tamaños grandes, el crecimiento sigue siendo acelerado conforme aumenta, ya que el algoritmo continúa recorriendo la parte restante del arreglo para localizar el elemento menor.

IV. Polinomio y Orden de Ejecución

```
void selectionSort(int arreglo[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int posicionMenor = i;
        for (int j = i + 1; j < n; j++) {
            if (arreglo[j] < arreglo[posicionMenor]) {
                posicionMenor = j;
            }
        }
        if (posicionMenor != i) {
            int auxiliar = arreglo[i];
            arreglo[i] = arreglo[posicionMenor];
            arreglo[posicionMenor] = auxiliar;
        }
    }
}
```

Mejor Caso	Peor Caso
$2n$	$2n$
$n - 1$	$n - 1$
$2n^2 + 2n - 2$	$n^2 + 2n - 2$
$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$
0	$\frac{n(n-1)}{2}$
0	0
0	0
$n - 1$	$n-1$
0	$n - 1$

<code>arreglo[i] = arreglo[posicionMenor];</code>	0	$n - 1$
<code>arreglo[posicionMenor] = auxiliar;</code>	0	$n - 1$
<code>}</code>		
<code>}</code>		
<code>}</code>		
	$2n^2 + 7n - 7$ $O(n^2)$	$\frac{3n^2 + 9n - 8}{2}$ $O(n^2)$

Conclusiones

Álvarez Tahuilán Luis Gustavo

A partir del análisis teórico y de los resultados obtenidos, se puede concluir que el algoritmo de Ordenamiento por Selección presenta un crecimiento cuadrático conforme aumenta el tamaño del arreglo. Esto se observa con mayor claridad a partir de tamaños grandes, ya que los tiempos pasan de fracciones de segundo para arreglos pequeños a varios minutos cuando se trabaja con 500,000 y 1,000,000 de elementos.

Aunque se utilizaron cuatro condiciones distintas: aleatorio, inverso, ordenado y casi ordenado, todas presentan una tendencia de crecimiento similar. Esto concuerda con el análisis analítico, ya que Selection Sort continúa recorriendo la parte restante del arreglo para localizar el elemento menor, incluso cuando los datos ya están ordenados. Por esta razón, tanto en el mejor como en el peor caso su orden de ejecución se mantiene en $O(n^2)$.

Los tiempos obtenidos no son exactamente proporcionales entre todas las pruebas, debido a factores propios de la ejecución en la computadora y a las diferencias en las operaciones realizadas para cada distribución. Sin embargo, conforme aumenta n , el comportamiento cuadrático se vuelve evidente, confirmando lo esperado teóricamente.

Noyola Gómez Emilio Damian

Los resultados experimentales permiten observar que Selection Sort funciona adecuadamente con arreglos pequeños, pero pierde eficiencia rápidamente al aumentar la cantidad de elementos. Para tamaños de 100 y 500 elementos el tiempo registrado fue prácticamente cero, mientras que para un arreglo de 1,000,000 de elementos los tiempos estuvieron entre aproximadamente 648 y 906 segundos, dependiendo de la condición inicial.

También se observó que el arreglo inverso presentó algunos de los tiempos más altos, llegando a 905.565 segundos para un millón de elementos, mientras que el arreglo casi ordenado obtuvo 647.758 segundos para el mismo tamaño. Sin embargo, que una condición sea más rápida que otra en una ejecución concreta no modifica la complejidad del algoritmo, ya que en todos los casos la cantidad de trabajo principal crece en función de n^2 .

La práctica permitió comprobar la diferencia entre el tiempo real de ejecución y el orden teórico de un algoritmo. Los valores experimentales pueden variar por las características de la computadora, el compilador y la medición, pero la tendencia general coincide con el análisis matemático: el Ordenamiento por Selección tiene un costo temporal cuadrático y resulta poco conveniente para conjuntos de datos de gran tamaño.

Bibliografía

- Brassard, G., & Bratley, P. (1997). *Fundamentos de algoritmia*. Prentice Hall.
- Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1998). *Estructuras de datos y algoritmos*. Pearson Educación/Addison Wesley.